

Introduction to uv

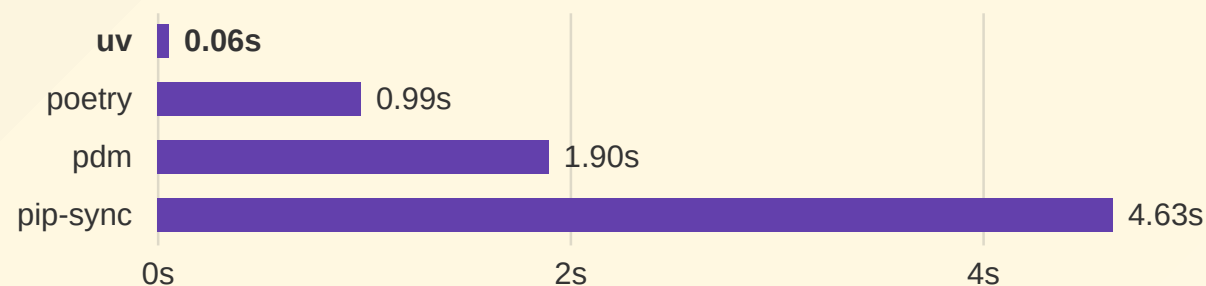
Outline

- Why uv?
- Installation and setup
- Scripts and tools
- Projects and dependencies
- References

What is uv?

An extremely fast Python package and project manager, written in Rust.

- 管理 Python 版本
- 建立與同步 project environment
- 執行 scripts 與 CLI tools
- 提供相容 pip 的介面



ref: <https://docs.astral.sh/uv/>

Astral

- Astral 是一間專注於 Python tooling 的公司
- 主要使用 Rust 開發高效能開發工具
- 旗下產品包含 uv, Ruff, ty, pyx(beta)
- Rye 已停止維護，官方建議遷移到 uv

ref: <https://astral.sh/>

回憶

- 先接觸 Ruff
- 後來觀望 uv
- uv 支援 project management 後開始使用



考古

- [2024-02-15](#): uv 作為 pip, pipx, venv 的替代品
- [2024-08-20](#): uv 加入 project management
- [2025-12-16](#): ty beta release
- [2026-03-19](#): OpenAI 宣布將收購 Astral; 完成後 Astral 將加入 Codex 團隊

ref: <https://astral.sh/blog>

印象中以前的習慣

- pip/venv + requirements.txt or setup.py
- conda + environment.yml
- poetry + pyproject.toml
- rye + pyproject.toml

當時為什麼改用 uv?

- 速度非常快
- 一次取代 pip, pipx, venv, pyenv, poetry 等工具
- 使用 cross-platform lockfile (`uv.lock`) 提升可重現性
- 可以管理 Python 版本
- 對 monorepo 支援很好 (`workspace`)

Installation

```
# macOS and Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"

# Windows (Scoop)
scoop install main/uv
```

正式環境可先檢查 installer，或改用 Homebrew、WinGet、pipx 等 package manager。

ref: <https://docs.astral.sh/uv/getting-started/installation/>

Upgrading uv

```
# Standalone installer
uv self update

# PyPI installation
pip install --upgrade uv
```

使用 package manager 安裝時，請改用該 package manager 的更新方式。

ref: <https://docs.astral.sh/uv/getting-started/installation/#upgrading-uv>

Shell autocompletion

```
# zsh
echo 'eval "$(uv generate-shell-completion zsh)"' >> ~/.zshrc

# fish
echo 'uv generate-shell-completion fish | source' > ~/.config/fish/completions/uv.fish
```

`uvx` 也有獨立 completion，可以用 `uvx --generate-shell-completion` 產生。

ref: <https://docs.astral.sh/uv/getting-started/installation/#shell-autocompletion>

Running script

```
uv run example.py

# Create a Python script with metadata
uv init --script example.py --python 3.12

# Add dependencies to the script
uv add --script example.py httpx

# Run the script
uv run --script example.py
```

ref:

<https://docs.astral.sh/uv/guides/scripts/>

example.py

```
# /// script
# dependencies = [
#     "requests<3",
#     "rich",
# ]
# ///

import requests
from rich.pretty import pprint

resp = requests.get("https://peps.python.org/api/peps.json")
data = resp.json()
pprint([(k, v["title"]) for k, v in data.items()][:10])
```

Using tools

```
uvx ruff check
```

```
# With a specific version
```

```
uvx ruff@0.3.0 check
```

```
# From a GitHub repo
```

```
uvx --from git+https://github.com/httpie/cli httpie
```

```
# Install and run a tool
```

```
uv tool install ruff
```

```
ruff check
```

ref: <https://docs.astral.sh/uv/guides/tools/>

Working on projects

```
# Create a new project
uv init hello-world
cd hello-world

# Initialize the current directory
mkdir hello-world
cd hello-world
uv init
```

uv 會使用 `pyproject.toml` 描述專案，並用 `uv.lock` 記錄可重現的解析結果。

ref: <https://docs.astral.sh/uv/guides/projects/>

Creating a project for a library

```
uv init --lib
```

```
.  
├── README.md  
├── pyproject.toml  
└── src  
    ├── hello_world  
    │   ├── __init__.py  
    │   └── py.typed
```

```
3 directories, 4 files
```

pyproject.toml (PEP 621)

```
[project]
name = "hello-world"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
authors = [
    { name = "narumi", email = "toucans-cutouts0f@icloud.com" }
]
requires-python = ">=3.14"
dependencies = []

[build-system]
requires = ["uv_build>=0.9.30,<0.10.0"]
build-backend = "uv_build"
```


Add packages

```
# Add a package to the current project
uv add httpx

# Add dev dependencies
uv add --dev ruff ty pytest pytest-cov

# Add a git dependency
uv add git+https://github.com/psf/requests
```

`uv add` 會更新 `pyproject.toml`，並重新解析 `uv.lock`。

ref: <https://docs.astral.sh/uv/guides/projects/#managing-dependencies>

Remove packages

```
# Remove a package from the current project  
uv remove httpx
```

```
# Remove dev dependencies  
uv remove --dev ruff ty
```

`uv remove` 會同步移除 dependency declaration 並更新 lockfile。

ref: <https://docs.astral.sh/uv/guides/projects/#managing-dependencies>

uv lock

```
uv lock
uv lock --check
uv lock --upgrade
uv lock --upgrade-package httpx
```

- `uv lock`: 建立或更新 `uv.lock`，不同步 `.venv`
- `uv lock --check`: 檢查 lockfile 是否符合 project metadata
- 新版本套件釋出不會自動讓 lockfile 過期
- 升級 locked versions 時要明確使用 `--upgrade`

ref: <https://docs.astral.sh/uv/concepts/projects/sync/>

Project workflow

```
uv init
uv add httpx
uv run python main.py
uv sync
```

- `uv sync`: 需要時更新 `uv.lock`，再同步 `.venv`
- `uv sync --locked`: lockfile 過期時 error，適合 CI
- `uv sync --frozen`: 直接使用現有 lockfile，不檢查 project metadata

ref: <https://docs.astral.sh/uv/concepts/projects/sync/>

Uninstallation

```
uv cache clean  
rm -r "$(uv python dir)"  
rm -r "$(uv tool dir)"  
rm ~/.local/bin/uv ~/.local/bin/uvx
```

移除前可先清 cache、Python installations 與 tools；Windows 與其他安裝方式請依官方文件操作。

ref: <https://docs.astral.sh/uv/getting-started/installation/#uninstallation>

AGENTS.md

- Follow project-defined tooling first.
- For Python work, use `uv` for execution and package management, `ruff` for formatting and linting, and `ty` for type checking.

ref:

<https://github.com/narumiruna/dotfiles/blob/main/codex/.codex/AGENTS.md>

pre-commit config

```
repos:  
  - repo: https://github.com/astral-sh/uv-pre-commit  
    rev: 0.11.7  
    hooks:  
      - id: uv-lock
```

ref: <https://github.com/astral-sh/uv-pre-commit>

References

- [Astral](#)
- [Astral blog](#)
- [OpenAI to acquire Astral](#)
- [uv docs](#)